

Document RAG System — Technical Architecture

End-to-End Chat Request Lifecycle, Multi-Stage Pipeline & Gemini API Call Analysis

Backend: ASP.NET Core 9.0

Vector Store: Qdrant

LLM: Google Gemini 3.6 Flash

Embeddings: gemini-001

1. Executive Summary & Core Architecture

The Document RAG System is a production multi-stage technical retrieval-augmented generation engine engineered specifically for industrial product documentation (ebm-papst ventilation systems, motors, and drive technology). The architecture combines deterministic domain heuristics with semantic vector search, specialized workflow routing, claim-level evidence evaluation, and strict output guardrails to guarantee factual grounding and eliminate hallucinations.

Core Architectural Building Blocks

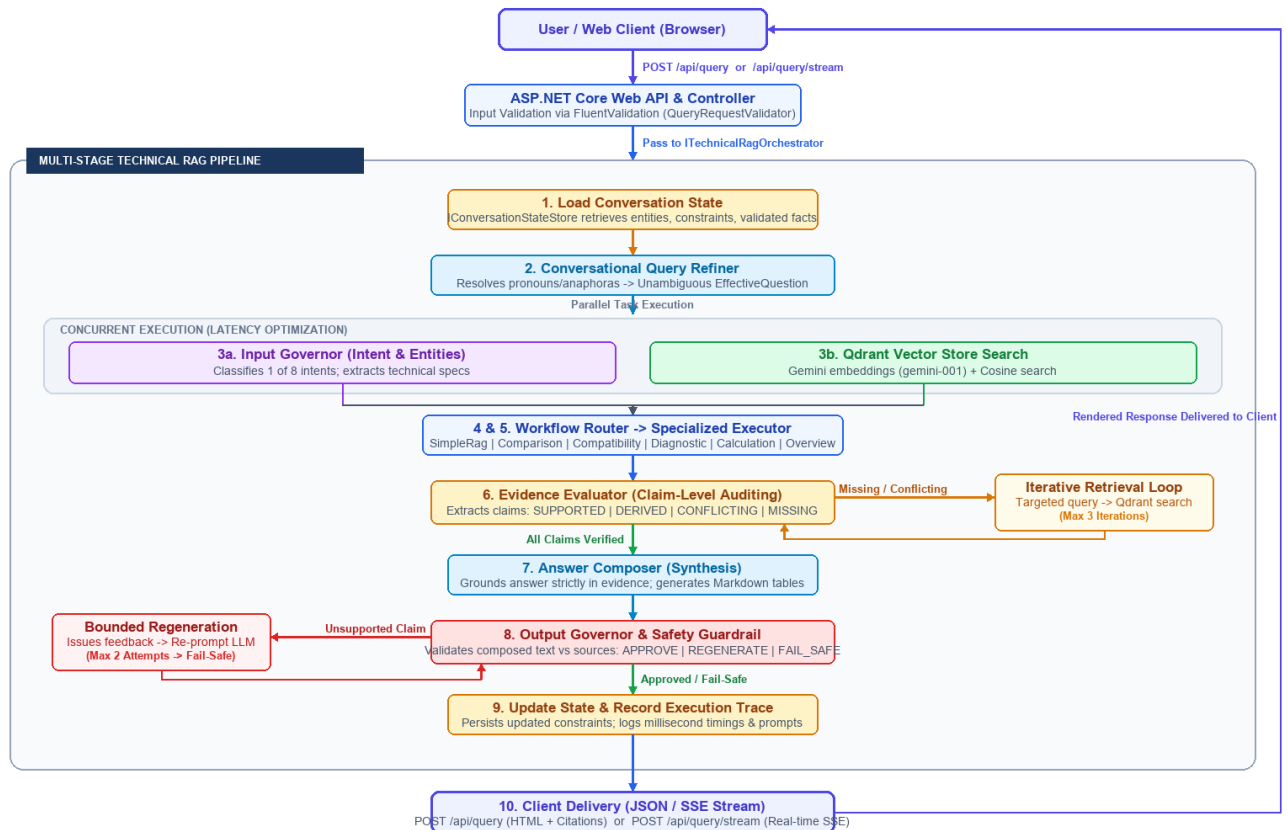
- **ASP.NET Core Web API:** Provides REST (POST /api/query) and Server-Sent Events (POST /api/query/stream) endpoints with FluentValidation.
- **TechnicalRagOrchestrator:** Coordinates conversational turn state, parallel execution, routing, evidence verification, and output safety.
- **ConversationalQueryRefiner:** Resolves multi-turn pronouns/anaphoras, tracks active constraints, and outputs an unambiguous EffectiveQuestion.
- **InputGovernor & WorkflowRouter:** Classifies query intent across 8 technical domains and routes to dedicated domain workflow executors.
- **Qdrant Vector Store & Gemini Embeddings:** Generates dense 768-dim embeddings via gemini-embedding-001 and executes cosine similarity searches.
- **EvidenceEvaluator & Iterative Retrieval:** Extracts claims from drafts (SUPPORTED, DERIVED, CONFLICTING, MISSING) and executes targeted queries for missing data.
- **AnswerComposer:** Synthesizes final responses in the user's language, generating markdown tables and specifications with citation links.
- **OutputGovernor:** Audits composed output against retrieved source context chunks with bounded regeneration loops to enforce safety.
- **IConversationStateStore:** Persists conversation entities, constraints, validated facts, and candidate sets across turns.

2. Architecture Flowchart: Chat Request Pipeline

The diagram below illustrates the end-to-end journey of a user chat request from initial HTTP ingestion to verified client delivery, highlighting the concurrent execution of governance & search and the two feedback loops:

Document RAG System — Chat Request Architecture & Pipeline

End-to-end flow from user HTTP/SSE request to verified technical response delivery



3. Detailed Request Lifecycle (The 10 Stages)

When a user executes a chat query, the system processes it through 10 distinct stages:

- **Stage 1: HTTP Ingestion & Validation:** Incoming payload (Question, ConversationId) is validated by QueryRequestValidator via FluentValidation. Invalid requests return HTTP 400 immediately.
- **Stage 2: State Retrieval:** IConversationStateStore loads the ConversationState for the active ConversationId, restoring past entities, active constraints, and verified technical facts.
- **Stage 3: Query Refinement:** ConversationalQueryRefiner resolves context and pronouns (e.g. 'What is its airflow?' -> 'What is the airflow for 4114 N/2 H6PU?'), yielding an unambiguous EffectiveQuestion.
- **Stage 4: Parallel Governance & Search:** InputGovernor (intent classification) and QdrantVectorStore.SearchAsync (query embedding + vector search) execute concurrently to minimize latency.
- **Stage 5: Workflow Routing & Execution:** WorkflowRouter selects one of 8 specialized executors (e.g., Compatibility runs retrofit rules; Comparison builds multi-product matrix; Overview lists catalog).
- **Stage 6: Evidence Evaluation Loop:** EvidenceEvaluator checks technical claims in the draft. If missing or conflicting claims exist, it formulates targeted vector queries (up to MaxRetrievalIterations = 3).
- **Stage 7: Answer Composition:** AnswerComposer builds the final structured response in the user's language, assembling comparison tables, formulas, and grounded citations.
- **Stage 8: Output Governance Loop:** OutputGovernor validates that every claim in the composed answer is supported by source chunks. If unsupported claims exist, triggers bounded regeneration (up to 2 attempts).
- **Stage 9: State Persistence & Tracing:** Updated entities, constraints, and facts are saved back to IConversationStateStore. Complete timing breakdown and prompt traces are logged in ExecutionTrace.
- **Stage 10: Client Delivery:** Markdown is converted to HTML via Markdig with CitationDto objects for POST /api/query, or streamed token-by-token via SSE on POST /api/query/stream.

The 8 Specialized Technical Workflow Executors

- * **SimpleRag:** Single-model technical specification lookups and installation procedures.
- * **Comparison:** Multi-model side-by-side attribute matrices with ebm-papst type key decoding.
- * **Compatibility:** Retrofit and replacement compatibility checks (electrical, aerodynamic, mechanical).
- * **Diagnostic:** Fault isolation, symptom-to-cause mapping, and documented troubleshooting steps.
- * **Calculation:** Formulas and parameter-driven math for airflow, power, and operating points.
- * **Overview / Catalog:** Deterministic catalog inventories grouped by fan family and technology.
- * **Design:** System design recommendations and configuration guidelines.
- * **Clarification:** Formulates targeted clarifying questions when requirements are ambiguous.

4. Gemini API Request Breakdown per User Chat

Each user query consumes a mixture of Google Gemini Embedding API calls (gemini-embedding-001) and Google Gemini Generative LLM calls (gemini-3.6-flash). The exact number depends on conversation depth, execution mode, and whether evidence/regeneration loops fire.

Pipeline Stage	Component	Happy Path (1-Turn)	Multi-Turn	Worst-Case (Case E)
Vector Search	GeminiEmbeddingService	1 Embedding	1 Embedding	4 Embeddings
Stage 3: Refinement	ConversationalQueryRefiner	0 (Heuristic)	1 LLM	1 LLM
Stage 4: Governance	InputGovernor	0-1 LLM	1 LLM	1 LLM
Stage 5: Workflow	WorkflowExecutor	1 LLM	1 LLM	1 LLM
Stage 6: Evidence	EvidenceEvaluator	1 LLM	1 LLM	7 LLMs (Eval+TQ+Draft)
Stage 7: Composition	AnswerComposer	1 LLM	1 LLM	1 LLM
Stage 8: Output Guard	OutputGovernor	1 LLM (0 on Stream)	1 LLM	5 LLMs (Val+Regen+Fail)
Total API Requests	Full Pipeline	4 - 5 Requests	7 Requests	21 Requests (Max)

Operational Scenario Comparison

- **Scenario A — Single-Turn Happy Path (POST /api/query):** Consumes 1 Embedding + 3-4 LLMs (Total: 4-5 calls). First-turn queries bypass Query Refinement via regex heuristic fast-path.
- **Scenario B — Streaming Query (POST /api/query/stream):** Consumes 1 Embedding + 2-3 LLMs (Total: 3-4 calls). Streams tokens directly to user with minimal Time-To-First-Token (TTFT).
- **Scenario C — Multi-Turn Conversational Query:** Consumes 1 Embedding + 6 LLMs (Total: 7 calls). QueryRefiner executes an LLM prompt to resolve pronouns and merge active entity constraints.
- **Scenario D — Deterministic Domain Workflows (Overview / Replacement):** Consumes 1-2 calls total. EbmProductCodeParser decodes product type keys deterministically, bypassing LLM generation for standard catalogs.
- **Scenario E — Worst-Case Bounded Execution:** Consumes 4 Embeddings + 17 LLMs (Total: 21 calls maximum). Triggered when multi-turn context, missing evidence claims, and output rejections occur simultaneously.

5. Deep-Dive: Case E Flowchart & Step-by-Step Accounting

Case E represents the theoretical upper bound (21 Gemini requests) when all 3 safety mechanisms fire: multi-turn resolution + all 3 iterative retrievals + 2 output regenerations + fail-safe exit.

Case E: Worst-Case Bounded Scenario (21 Gemini Requests Breakdown)

Simultaneous multi-turn context + 3 iterative vector retrievals + 2 regeneration attempts -> Fail-Safe exit



GRAND TOTAL: 4 EMBEDDING CALLS + 17 GENERATIVE LLM CALLS = 21 GEMINI REQUESTS

Strict Mathematical Bounds: MaxRetrievalIterations = 3 | MaxRegenerationAttempts = 2 | Zero Unverified Claims

Phase	Sub-Step / Component	Operation Description	Gemini API Type	Calls
Phase 1	ConversationalQueryRefiner	Resolve multi-turn constraints & pronouns	LLM (JSON)	1
Phase 1	InputGovernor	Classify intent, extract entities & constraints	LLM (JSON)	1
Phase 1	GeminiEmbeddingService	Generate vector embedding for initial Qdrant search	Embedding	1
Phase 2	WorkflowExecutor	Generate initial draft answer from context chunks	LLM (Text/JSON)	1
Phase 3	EvidenceEvaluator (Iter 1-3)	Evaluate claims against context (3 iterations)	LLM (JSON)	3
Phase 3	Targeted Query Gen (Iter 1-3)	Generate keyword query for missing claim	LLM (Text)	3
Phase 3	Targeted Search (Iter 1-3)	Embed targeted query for supplementary Qdrant search	Embedding	3
Phase 3	Intermediate Redrafts (Iter 1-3)	Re-draft response with expanded chunk set	LLM (Text)	3
Phase 4	AnswerComposer	Synthesize structured final response from verified claims	LLM (Text)	1
Phase 5	OutputGovernor (Attempts 1-2)	Audit proposed answer for unsupported claims	LLM (JSON)	2
Phase 5	Regeneration Prompts (1-2)	Rewrite answer resolving Output Governor issues	LLM (Text)	2
Phase 5	Fail-Safe Generator	Generate polite fallback response directing to tech support	LLM (Text)	1
Total	Worst-Case Execution Upper Bound	4 Embedding Calls + 17 Generative LLM Calls	All APIs	21

Engineering & Safety Guardrail Justification

In industrial engineering, parameters such as voltage (230V vs 400V), airflow direction (A vs V), and physical cutout dimensions carry physical equipment risk if hallucinated. The system enforces two hard mathematical limits:

- **MaxRetrievalIterations = 3:** Prevents infinite retrieval loops when information is not in the database.
- **MaxRegenerationAttempts = 2:** Exits gracefully to FAIL_SAFE (advising the user to contact technical support) rather than presenting ungrounded guesses.